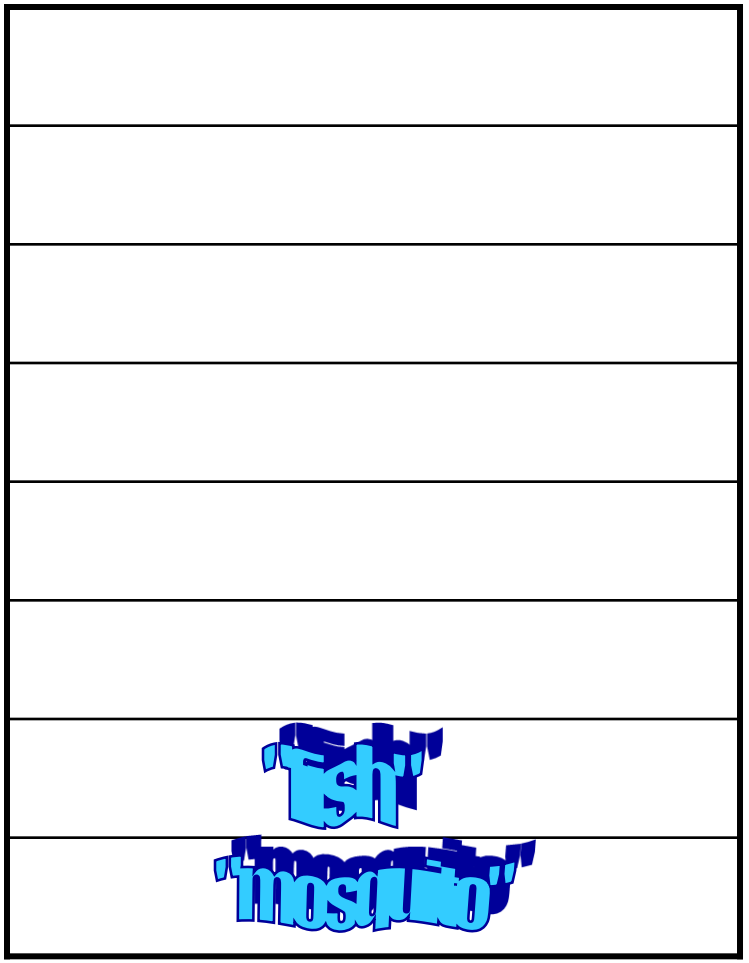


Stacks and Queues
Sections 3.6 and 3.7

Stack Model—LIFO

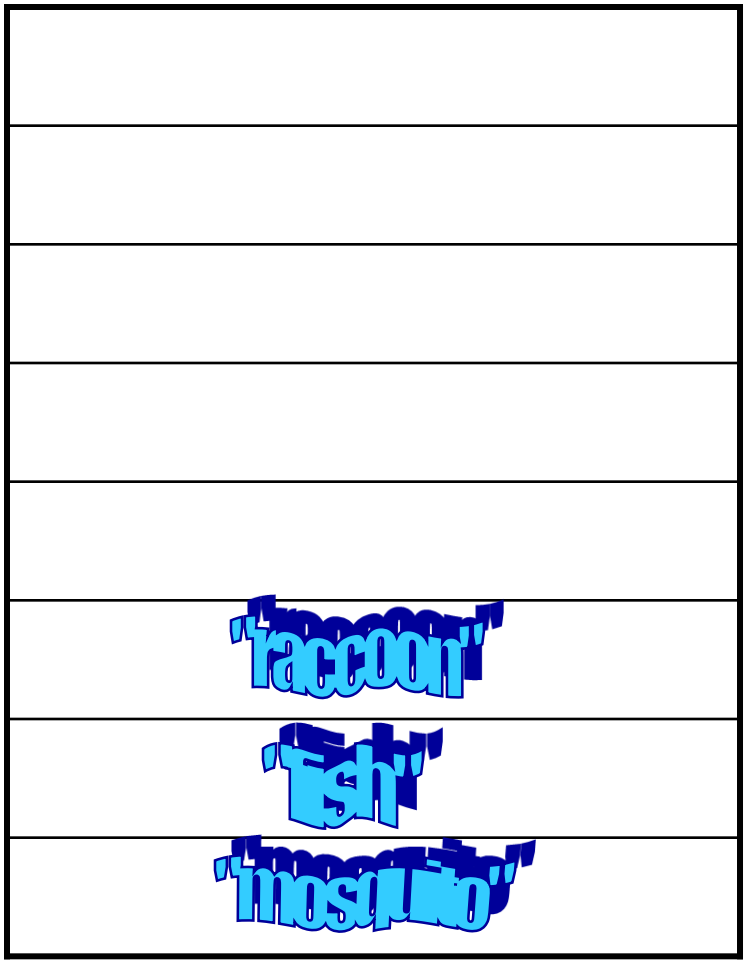
- Empty stack **S**
  - **S.empty()** is **true**
  - **S.top()** not defined
  - **S.size() == 0**
- **S.push("mosquito")**
  - **S.empty()** is **false**
  - **S.top() == "mosquito"**
  - **S.size() == 1**
- **S.push("fish")**
  - **S.empty()** is false
  - **S.top() == "fish"**
  - **S.size() == 2**



food chain stack

Stack Model—LIFO

- `S.push("raccoon")`
  - `S.empty()` is `false`
  - `S.top() == "raccoon"`
  - `S.size() == 3`
- `S.pop()`
  - `S.empty()` is `false`
  - `S.top() == "fish"`
  - `S.size() == 2`



food chain stack

Stack ADT

- Collections:
  - Elements of some proper type **`T`**
- Operations:
  - **`void push(T t)`**
  - **`void pop()`**
  - **`T top()`**
  - **`bool empty()`**
  - **`unsigned int size()`**
  - constructor and destructor

Uses of ADT Stack

- Backtracking for puzzles
- Balancing Symbols [()] is legal, but [(]) is wrong
- Evaluating postfix expressions (reverse polish calculator) **4.99 * 1.06 + 5.99 + 6.99 * 1.06 (infix)**

is stacked as : 4.99 1.06 * 5.99 + 6.99 1.06 * + (postfix)

- Function calls (runtime stack)
- Recursion using stack frame

Evaluating Postfix Expressions (Reverse Polish Calculator)

- Postfix expressions: operands precede operator
- Tokens: atomics of expressions, either operator or operand
- Example:
  - z = 25 + x*(y – 5)
  - Tokens: z, =, 25, +, x, *, (, y, -, 5,)

Evaluating Postfix Expressions (2)

- Evaluation algorithm:
  - Use stack of tokens
  - Repeat
    - If operand, push onto stack
    - If operator
      - pop operands off the stack
      - evaluate operator on operands
      - push result onto stack
    - Until expression is read
    - Return top of stack

Evaluating Postfix Expressions (3)

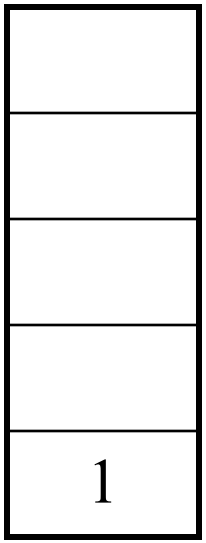
- Most CPUs have hardware support for this algorithm
- Translation from infix to postfix also uses a software

Evaluating Postfix Expressions (4)

- Original expression: 1 + (2 + 3) * 4 + 5
- Evaluate: 1 2 3 + 4 * + 5 +

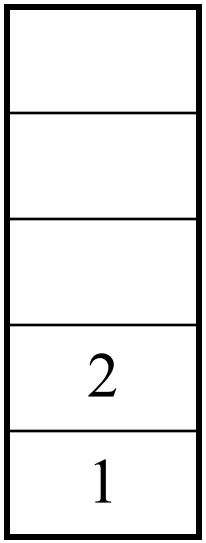
Evaluating Postfix Expressions (5)

- Input: 1 2 3 + 4 * + 5 +
- Push(1)



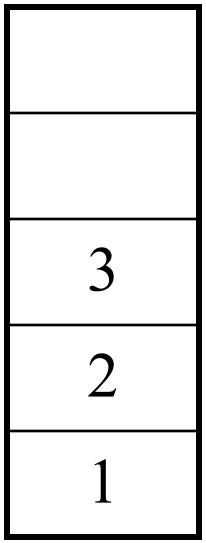
Evaluating Postfix Expressions (6)

- Input: 2 3 + 4 * + 5 +
- Push(2)



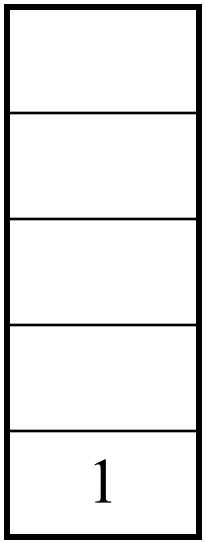
Evaluating Postfix Expressions (7)

- Input: 3 + 4 * + 5 +
- Push(3)



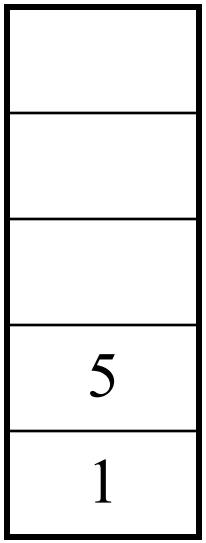
Evaluating Postfix Expressions (8)

- Input: + 4 * + 5 +
- Pop() == 3
- Pop() == 2



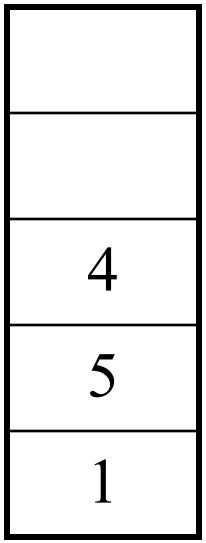
Evaluating Postfix Expressions (9)

- Input: + 4 * + 5 +
- Push(2 + 3)



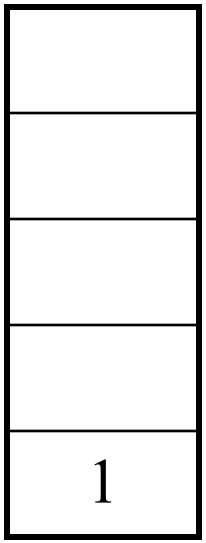
Evaluating Postfix Expressions (10)

- Input: 4 * + 5 +
- Push(4)



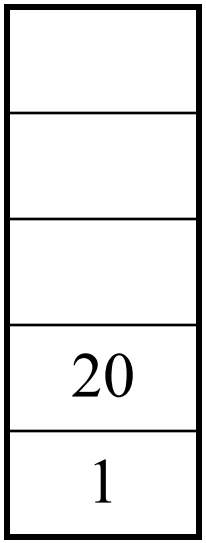
Evaluating Postfix Expressions (11)

- Input: * + 5 +
- Pop() == 4
- Pop() == 5



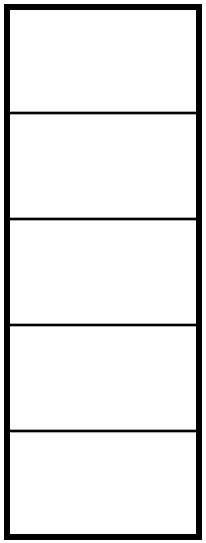
Evaluating Postfix Expressions (12)

- Input: * + 5 +
- Push(5 * 4)



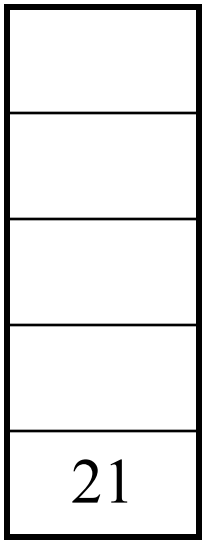
Evaluating Postfix Expressions (13)

- Input: + 5 +
- Pop() == 20
- Pop() == 1



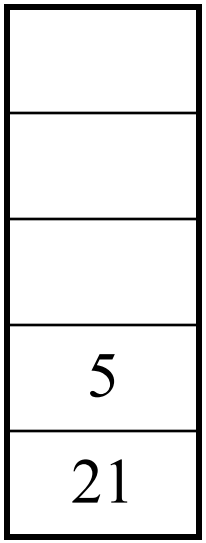
Evaluating Postfix Expressions (14)

- Input: + 5 +
- Push(1 + 20)



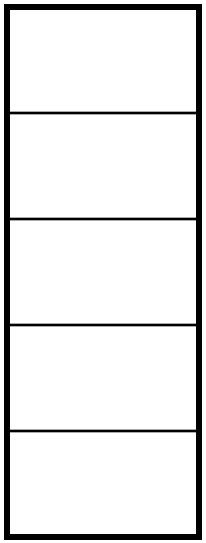
Evaluating Postfix Expressions (15)

- Input: 5 +
- Push(5)



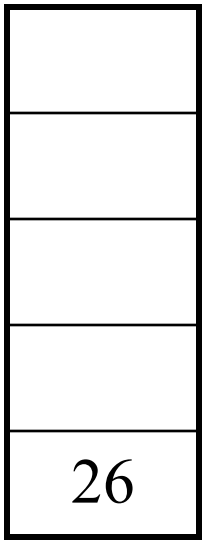
Evaluating Postfix Expressions (16)

- Input: +
- Pop() == 21
- Pop() == 5



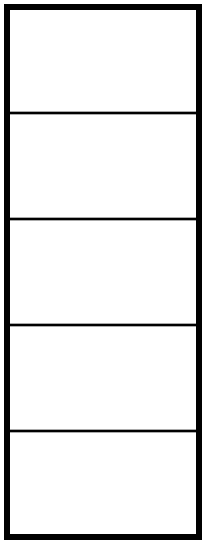
Evaluating Postfix Expressions (17)

- Input: +
- Push(21 + 5)



Evaluating Postfix Expressions (18)

- Input:
- Pop() == 26



Postfix Evaluation Implementation

```
Evaluate(postfix expression) {

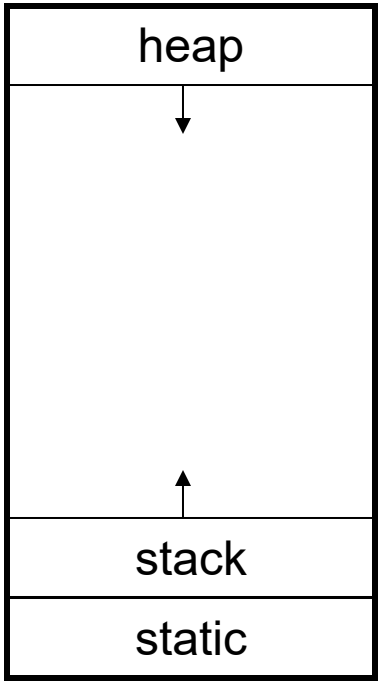
   // use stack of tokens;
   while(// expression is not empty) {
        t = next token;
        if (t is operand) {
               // push onto stack
        } else {
               // pop operands for t off stack
               // evaluate t on these operands
               // push result onto stack
        }
   }
   // return top of stack
}
```

Exercise

- How to use stack to balance brackets

Runtime Stack

- Runtime environment
  - Static/ Data
    - Global variables
    - Static variables
  - Code
    - Executable code
  - Stack
    - Push for each function call
    - Pop for each function return
  - Heap
    - Dynamic new and delete



program memory

Recursion

- Order 1: (Direct)
  - function calls itself
- Order 2: (Indirect)
  - `f()` calls `g()`, and `g()` calls `f()`
- Facilitated by stack

Recursion (3!)

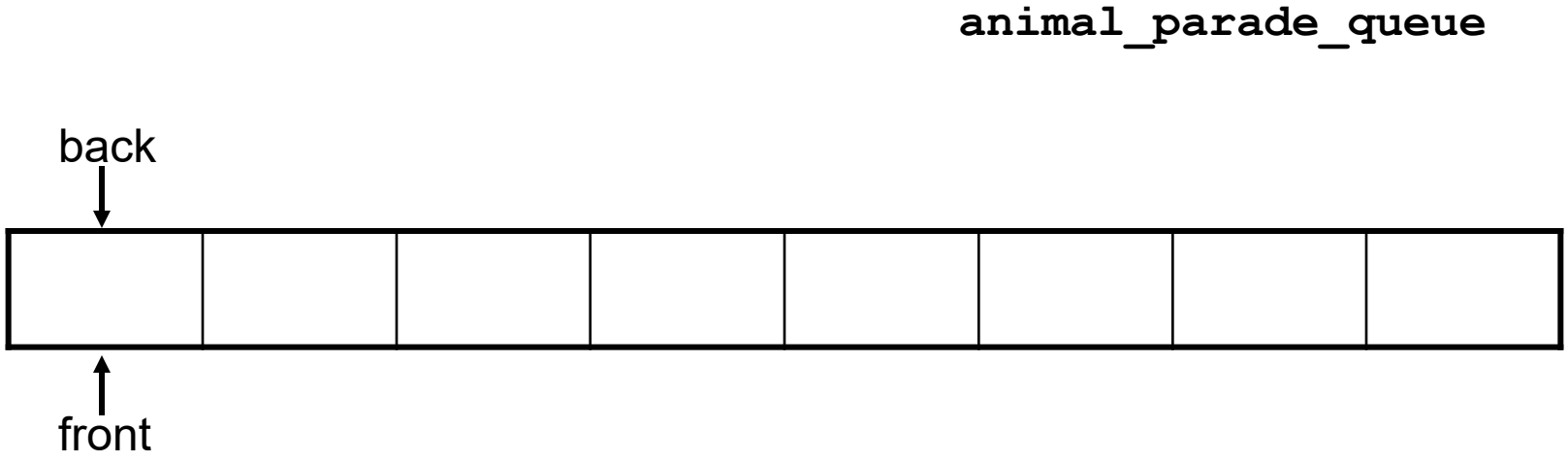
Step	Action	Function Call	Stack Contents (Top → Bottom)	Returned Value / Evaluation
1	Call	`factorial(3)`	[factorial(3)]	waiting for `factorial(2)`
2	Call	`factorial(2)`	[factorial(2), factorial(3)]	waiting for `factorial(1)`
3	Call	`factorial(1)`	[factorial(1), factorial(2), factorial(3)]	waiting for `factorial(0)`
4	Call	`factorial(0)`	[factorial(0), factorial(1), factorial(2), factorial(3)]	base case reached
5	Return	`factorial(0)` returns 1	[factorial(1), factorial(2), factorial(3)]	result = 1
6	Return	`factorial(1)` = 1 × 1	[factorial(2), factorial(3)]	result = 1
7	Return	`factorial(2)` = 2 × 1	[factorial(3)]	result = 2
8	Return	`factorial(3)` = 3 × 2	[ ] (empty)	result = 6

Stack Adaptor Requirements

- Stack
  - `push()`
  - `pop()`
  - `top()`
  - `empty()`
  - `size()`
- Can use List, Array

Queue Model—FIFO

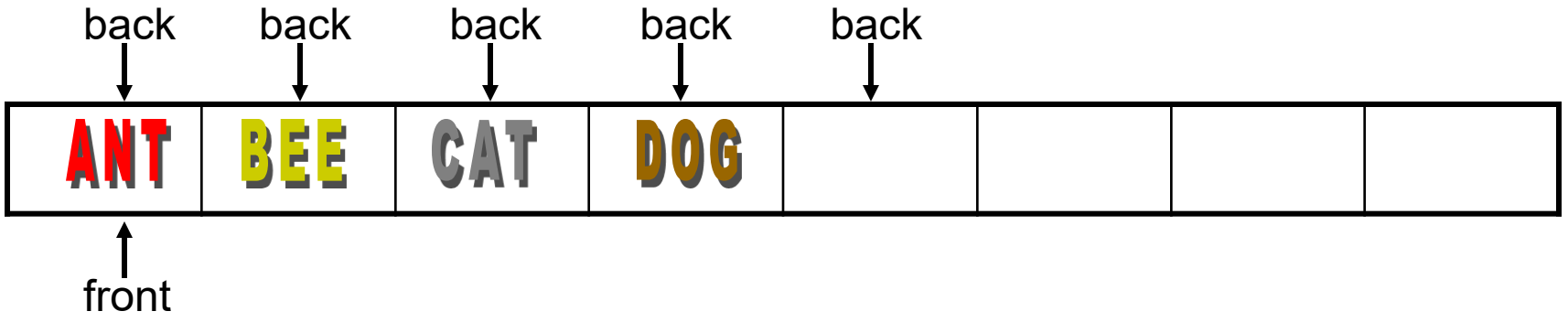
- Empty Q



Queue Model—FIFO

- `Q.push( "ant" )`
- `Q.push( "bee" )`
- `Q.push( "cat" )`
- `Q.push( "dog" )`

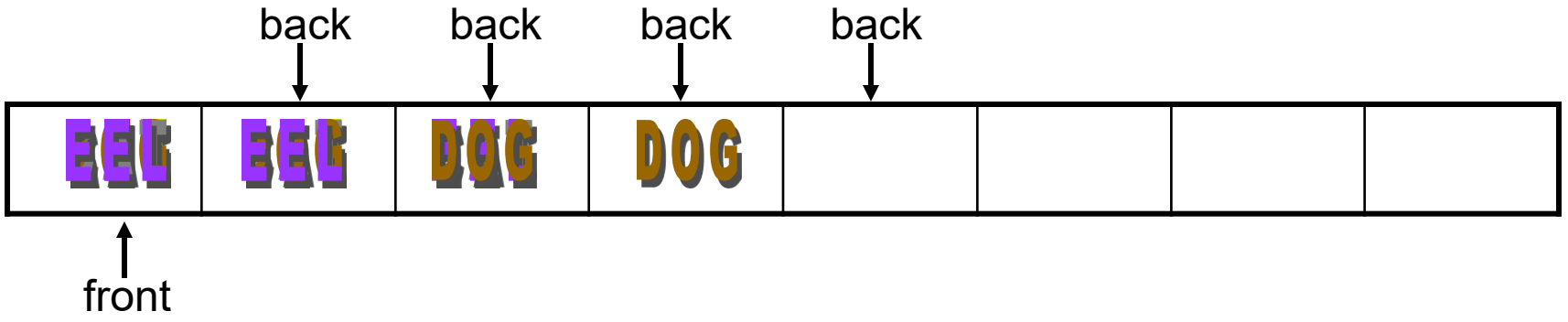
`animal_parade_queue`



Queue Model—FIFO

- `Q.pop();`
- `Q.pop();`
- `Q.push( "eel" );`
- `Q.pop();`
- `Q.pop();`

`animal_parade_queue`



Queue ADT - FIFO

- Collection
  - Elements of some proper type **`T`**
- Operations
  - **`void push(T t)`**
  - **`void pop()`**
  - **`T front()`**
  - **`T back()`**
  - **`bool empty()`**
  - **`unsigned int size()`**
  - Constructors and destructors

Uses of ADT Queue

- Buffers (storage areas where data waits to be processed like in networking)
- Breadth first search
- Simulations (like in customer service lines, traffic simulations)
- Producer-Consumer Problems (Producers enqueue while consumers dequeue)

Queue Adaptor Requirements

- Queue
  - `push()`
  - `pop ()`
  - `front()`
  - `back()`
  - `empty()`
  - `size()`
- Can use List, Array

Class Stack

```
template <typename T, class Container>

class Stack {
   protected:
        Container c;

   public:
        void push(const T & x) { c.push_back(x); }
        void pop() { c.pop_back(); }
        T top() const { return c.back(); }
        int empty() const { return c.empty(); }
        unsigned int size() const { return c.size(); }
        void clear() { c.clear(); }
};
```

Declarations

- `Stack< float, List<float>> floatStack;`
- `Stack< int, Vector<int> > intStack;`

Class Queue

```
template <typename T, class Container>
class Queue {
  protected:
      Container c;

  public:

      void push(const T & x) { c.push_back(x); }
      void pop() { c.pop_front(); }
      T front() const { return c.front(); }
      int empty() const { return c.empty(); }
      unsigned int size() const { return c.size(); }
      void clear() { c.clear(); }
};
```